

QIntel: A Database-Driven Question Management System for Events

Susan Mohammad

Course: 6354 - Information Management – Spring 2026

Professor: Karl Ho

Application link : https://qevent.shinyapps.io/qevent_app/

Table of Contents

Problem:.....	2
Introduction:	2
System Requirements:.....	2
Technical requirements:	2
Technology used:	2
Star Schema Design	3
ER Diagram.....	3
AI Integration:.....	5
Challenges and Lessons learned	7
Spelling challenge.....	7
Country detection challenge.....	8
Prompt to AI models	9
Pictures of the application:	10
Appendix.....	11

Problem:

Always there are some chaos in the event for asking questions from people in the panel in events or when politicians are in the room with press, there is not enough chance for all press members to ask all of their questions from politicians. This results in dissatisfaction for those reporters who could not ask their questions and for speaker as they are facing bullying by some reporters and cannot answer other reporters' questions.

To help address this issue I created this application so that way all reporters have a fair chance to put all of their questions on the platform. And the speaker has a chance to select best and most important and related questions. With this application, the time for both group will be saved, the chaos in meetings, especially the important ones, will disappear.

Introduction:

The QIntel is a specialized system built with R and Shiny for event logic and response tracking. It utilizes a country detection algorithm and a SQLite backend to manage participant questions in real-time. It allows speakers to filter, classify, and display audience questions during live events.

System Requirements:

In this application we need an Admin who define the event's name, date and location, a Reporter or Participant who submit questions, with their name and the organization that they are coming from to the event, and a Speaker who select questions, approve them and answer them and in the display page select next question to answer to questions one by one.

Technical requirements:

For the AI model functionality, we needed a better LLM from Cloud to do the tokenization for topic modeling but as we did not want to pay for tokenization and cloud we used Quanteda and LDA. With the fuzzy function we identify country that they are typing in the question part (with typo, but it is not very smart to identify the country from a question while they did not bring the name of any country in the question). There is a speech to text option in the last page, when the speaker select a question can click on the speaker button and start speaking, then it types all the words.

Technology used:

Component	Technology
Frontend/Backend	Shiny (R)
Database	SQLite with star schema
Topic Modeling	Quanteda + Latent Dirichlet Allocation (LDA) (topic models)
Location Detection	Fuzzy string matching (adist)
Spell Checking	hunspell
Map Data	maps package (iso3166)
Voice Input	Web Speech API

Star Schema Design

In this project I needed a core and important table which is fact_question

Entities:

dim_countries: for saving the country name, region and country id

dim_states: state_id, state_name, state_code(text), country_id

dim_cities: city_id, city_name, country_id, city_id

dim_events: saving event_id, event_name, date_id, location, country_id

dim_date: date_id, month, day, year

dim_participants: person_id, first_name, last_name, country_id, organization

dim_topic: topic_id, topic_name, policy_area

fact_questions: question_id, person_id, question_text, created_at, topic_id, event_id, answer_text, date_id, country_id, status

dim_answers: answer_id, question_id, responder, answer_text, answered_at

embeddings: question_id, embedding

question_clusters: cluster_id, main_question, answer_text, created at

ER Diagram

DB Browser for SQLite - C:\Users\smvic\Documents\Knowledge Mining\qevent_app\qevent_database.sqlite

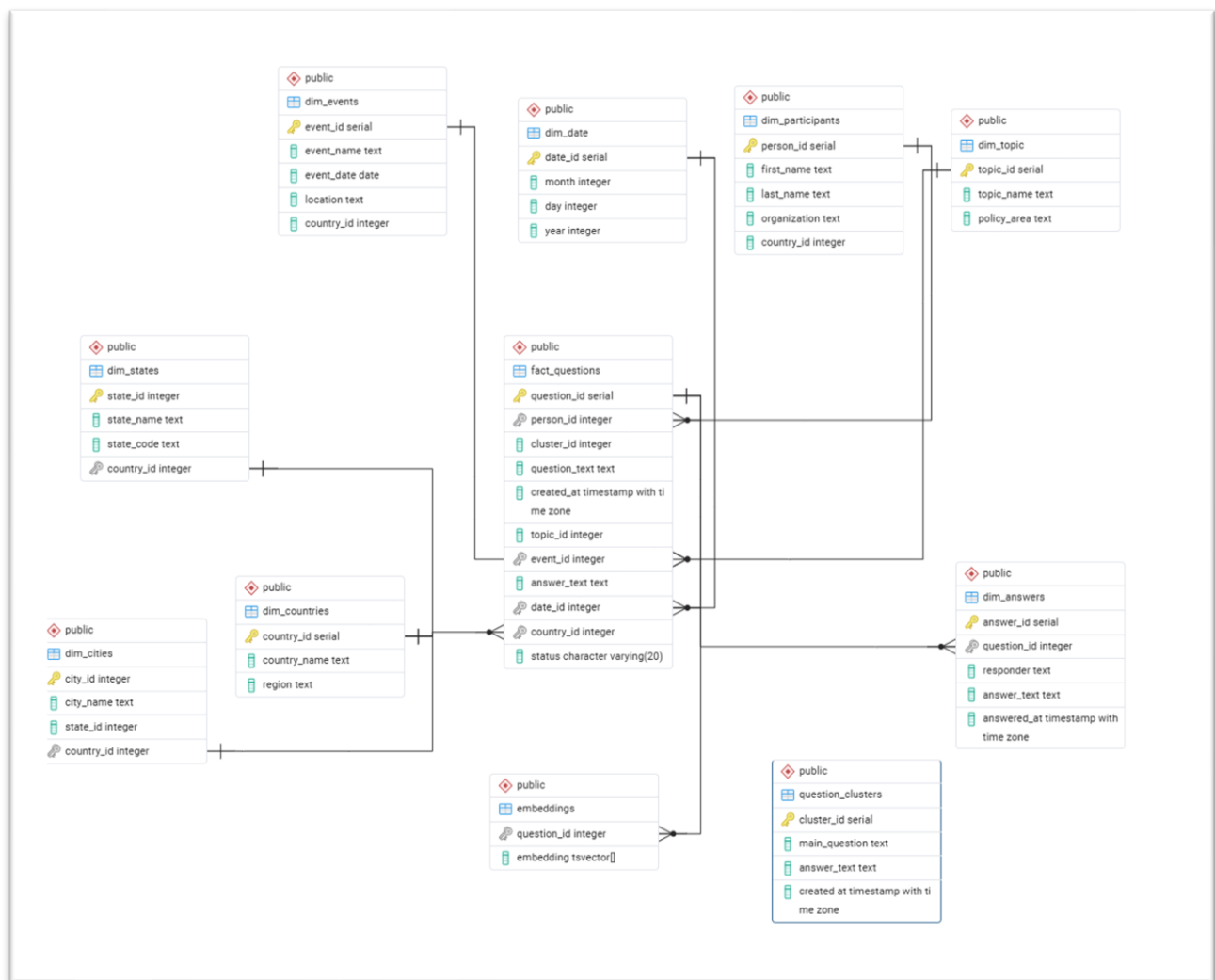
File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Create Table Create Index Modify Table Delete Table Print Refresh

Name	Type	Schema
Tables (11)		
dim_answers		CREATE TABLE dim_answers (answer_id INTEGER PRIMARY KEY, question_id INTEGER, responder TEXT, answer_text TEXT, answered_at TIMESTAM
dim_cities		CREATE TABLE dim_cities (city_id INTEGER PRIMARY KEY, city_name TEXT, state_id INTEGER, country_id INTEGER, FOREIGN KEY(state_id) REFER
dim_countries		CREATE TABLE dim_countries (country_id INTEGER PRIMARY KEY, country_name TEXT UNIQUE, region TEXT)
dim_date		CREATE TABLE dim_date (date_id INTEGER PRIMARY KEY, day INTEGER, month INTEGER, year INTEGER)
dim_events		CREATE TABLE dim_events (event_id INTEGER PRIMARY KEY, event_name TEXT, location TEXT, country_id INTEGER, date_id INTEGER, FOREIGN I
dim_participants		CREATE TABLE dim_participants (person_id INTEGER PRIMARY KEY, first_name TEXT, last_name TEXT, organization TEXT, country_id INTEGER, FC
dim_states		CREATE TABLE dim_states (state_id INTEGER PRIMARY KEY, state_name TEXT, state_code TEXT, country_id INTEGER, FOREIGN KEY(country_id) F
dim_topic		CREATE TABLE dim_topic (topic_id INTEGER PRIMARY KEY, topic_name TEXT, policy_area TEXT)
embeddings		CREATE TABLE embeddings (question_id INTEGER PRIMARY KEY, embedding TEXT, FOREIGN KEY(question_id) REFERENCES fact_questions(questi
fact_questions		CREATE TABLE fact_questions (question_id INTEGER PRIMARY KEY, person_id INTEGER, cluster_id INTEGER, question_text TEXT, created_at TIME
question_clusters		CREATE TABLE question_clusters (cluster_id INTEGER PRIMARY KEY, main_question TEXT, answer_text TEXT, created_at TIMESTAMP)
Indices (0)		
Views (0)		
Triggers (0)		



AI Integration:

Topic detection with LLM:

I used Quanteda and LDA to identify topic in the question. That way it can submit a specific topic related to the question. It was very challenging because firstly I needed that it identify the country name and do topic modeling automatically and when I asked AI model, it suggested me LLM embedding model that was based on cloud and it was not a free model, so I had to change my prompt to ask for another model which is free. It is still has a problem because it is on the local host and it cannot do the topic modeling on the web. And I asked several other AI model like deepseek and chat GPT to find a one on the web but I could not find a free model.

```
224 # =====
225
226 train_topic_model <- function(con) {
227   questions <- dbGetQuery(
228     con,
229     "SELECT question_text
230      FROM fact_questions
231      WHERE question_text IS NOT NULL AND question_text != ''
232   )
233
234   if (nrow(questions) < 3) {
235     message("Need at least 3 questions for LDA. Currently have: ", nrow(questions))
236     return(NULL)
237   }
238
239   toks <- tokens(
240     questions$question_text,
241     remove_punct = TRUE,
242     remove_numbers = TRUE,
243     remove_symbols = TRUE
244   )
245   toks <- tokens_tolower(toks)
246   toks <- tokens_remove(
247     toks,
248     c(stopwords("en"), "question", "questions", "problem", "issue", "test")
249   )
250
251   # Remove tokens shorter than 3 characters
252   toks <- tokens_select(toks, min_nchar = 3)
253
254   dfmat <- dfm(toks)
255   dfmat <- dfm_trim(dfmat, min_docfreq = 2)
256
257   if (nfeat(dfmat) < 2) {
258     message("Not enough features for LDA")
259     return(NULL)
260   }
261
262   dtm <- convert(dfmat, to = "topicmodels")
263
264   lda_model <- LDA(
265     dtm,
266     k = 3,
267     control = list(seed = 1234)
268   )
269
270   message("LDA Model trained successfully with ", nfeat(dfmat), " features")
271
272   list(
273     model = lda_model,
```

Predict function

```

278 predict_topic <- function(question_text, lda_data) {
279   if (is.null(lda_data)) {
280     return("General")
281   }
282   if (is.null(question_text) || nchar(trimws(question_text)) < 5) {
283     return("General")
284   }
285   toks <- tokens(
286     question_text,
287     remove_punct = TRUE,
288     remove_numbers = TRUE,
289     remove_symbols = TRUE
290   )
291   toks <- tokens_tolower(toks)
292   toks <- tokens_remove(
293     toks,
294     c(stopwords("en"), "question", "questions", "problem", "issue", "test")
295   )
296   toks <- tokens_select(toks, min_nchar = 3)
297   if (ntoken(toks) == 0) {
298     return("General")
299   }
300   new_dfm <- dfm(toks)
301   new_dfm <- dfm_match(new_dfm, features = lda_data$features)
302   if (sum(new_dfm) == 0 || ncol(new_dfm) == 0) {
303     return("General")
304   }
305   dtm_new <- convert(new_dfm, to = "topicmodels")
306   if (nrow(dtm_new) == 0 || ncol(dtm_new) == 0) {
307     return("General")
308   }
309   topic_probs <- posterior(lda_data$model, dtm_new)$topics
310   topic_num <- which.max(topic_probs[, 1])
311   top_terms <- terms(lda_data$model, 3)
312   if (!is.null(top_terms) && ncol(top_terms) >= topic_num) {
313     label <- paste(top_terms[, topic_num], collapse = " / ")
314   } else {
315     label <- paste("Topic", topic_num)
316   }
317 }

```

Fuzzy Location Detection

I used fuzzy function Levenshtein distance (adist function) to identify the typo with $\text{dist} \leq 2$. When I type a country name wrong it detect in the database the right name of the country.

I used library(map) as well. For UK, USA, and UAE that has different typing, I had to add some more codes that way it detect these countries. I also wanted that when I am typing it suggest me the right dictation so I used Hunspell library as well.

```

430 words <- unlist(strsplit(txt, "\\s+"))
431 best_dist <- Inf
432 best_match <- NULL
433 for (word in words) {
434   if (nchar(word) < 4) next
435   if (word %in% us_states) next
436   for (i in seq_len(nrow(all_countries))) {
437     cname <- tolower(all_countries$country_name[i])
438     d <- adist(word, cname)
439     if (d < best_dist && d <= 2) {
440       best_dist <- d
441       best_match <- all_countries[i, ]
442     }
443   }
444 }
445 if (!is.null(best_match)) {
446   return(list(
447     country_id = best_match$country_id,
448     country_name = best_match$country_name
449   ))
450 }
451 list(

```

```

detect_state_fuzzy <- function(question_text, con) {
  all_states <- dbGetQuery(
    con,
    "SELECT state_id, state_name, state_code FROM dim_states"
  )
  if (nrow(all_states) == 0) return(NA)
  question_lower <- tolower(question_text)
  for (i in seq_len(nrow(all_states))) {
    pattern_name <- paste0("\\b", tolower(all_states$state_name[i]), "\\b")
    pattern_code <- paste0("\\b", tolower(all_states$state_code[i]), "\\b")
    if (grep(pattern_name, question_lower) || grep(pattern_code, question_lower)) {
      return(list(
        state_id = all_states$state_id[i],
        state_name = all_states$state_name[i],
        state_code = all_states$state_code[i]
      ))
    }
  }
  words <- unlist(strsplit(question_lower, "\\s+"))
  best_match <- NULL
  best_dist <- Inf
  for (word in words) {
    # =====
    # SPELLING SUGGESTIONS
    # =====
    get_spelling_suggestions <- function(word) {
      if (nchar(word) < 3) return(character(0))
      clean_word <- gsub("[[:punct:]]", "", word)
      suggestions <- hunspell_suggest(clean_word)
      if (length(suggestions) > 0 && length(suggestions[[1]]) > 0) {
        return(suggestions[[1]][1:min(3, length(suggestions[[1]]))])
      }
      character(0)
    }
  }
}

```

Challenges and Lessons learned

I wanted that the speaker can select multiple questions and approve them then in the display page answering one by one and going to next question. While first time it brought all of the approved questions in one place, so I learned that I should create queue for questions. So firstly she select the questions are queued then approved and go to display. I understood that I need to define a flow for my application and regarding that flow defining code like the flow.

Spelling challenge

Another challenge was correction of typo, firstly I thought that when I run the app it will works on the local view while it did not work and I work a lot on the codes. However the problem was not related to codes (which it was a little challenging), it was the local view the typo detection is working on the shiny app web browser not the local view of R application.

```

# =====
# SPELLING SUGGESTIONS
# =====

get_spelling_suggestions <- function(word) {
  if (nchar(word) < 3) return(character(0))
  clean_word <- gsub("[[:punct:]]", "", word)
  suggestions <- hunspell_suggest(clean_word)
  if (length(suggestions) > 0 && length(suggestions[[1]]) > 0) {
    return(suggestions[[1]][1:min(3, length(suggestions[[1]]))])
  }
  character(0)
}

```

Country detection challenge

Firstly I added the name of the countries into my database PgAdmin, then every time I received errors when I wrote the name of some countries wrongly. I had many issues with the typos in country name to solve them which was one of the hardest part of my work. I wanted that AI find the country even with typo but I could not find a correct code for that, so I used fuzzy matching model with distance=2 to match the country that we write in the question with the current countries that I wrote in the database. Also when I am typing the question when I have typo and correct it, the problem will be solved.

```
# =====  
# COUNTRY / STATE / CITY DETECTION  
# =====  
  
detect_country_fuzzy <- function(question_text, con) {  
  all_countries <- dbGetQuery(  
    con,  
    "SELECT country_id, country_name FROM dim_countries"  
  )  
  
  txt <- tolower(question_text)  
  txt_clean <- gsub("[[:punct:]]", " ", txt)  
  
  # =====  
  # SPECIAL CASE: Common typos for multi-word countries  
  # =====  
  
  # United Arab Emirates  
  if (grepl("united arab emirat", txt_clean) || grepl("uae", txt)) {  
    uae <- all_countries[all_countries$country_name == "United Arab Emirates", ]  
    if (nrow(uae) > 0) {  
      return(list(country_id = uae$country_id[1], country_name = "United Arab Emirates"))  
    }  
  }  
  
  # United States  
  if (grepl("united states|usa|u s a", txt)) {  
    us <- all_countries[all_countries$country_name == "United States", ]  
    if (nrow(us) > 0) {  
      return(list(country_id = us$country_id[1], country_name = "United States"))  
    }  
  }  
  
  # United Kingdom  
  if (grepl("united kingdom|uk|u k", txt)) {  
    uk <- all_countries[all_countries$country_name == "United Kingdom", ]  
    if (nrow(uk) > 0) {  
      return(list(country_id = uk$country_id[1], country_name = "United Kingdom"))  
    }  
  }  
  
  # =====  
  # US STATES: If found, return United States immediately  
  # =====  
  
  us_states <- c(  
    "alabama", "alaska", "arizona", "arkansas", "california", "colorado",  
    "connecticut", "delaware", "florida", "georgia", "hawaii", "idaho",  
    "illinois", "indiana", "iowa", "kansas", "kentucky", "louisiana",  
    "maine", "maryland", "massachusetts", "michigan", "minnesota",  
    "mississippi", "missouri", "montana", "nebraska", "nevada",  
    "new hampshire", "new jersey", "new mexico", "new york",  
    "north carolina", "north dakota", "ohio", "oklahoma", "oregon",  
    "pennsylvania", "rhode island", "south carolina", "south dakota",  
    "tennessee", "texas", "utah", "vermont", "virginia", "washington",  
    "west virginia", "wisconsin", "wyoming"  
  )  
  
  for (state in us_states) {  
    if (grepl(paste0("\\b", state, "\\b"), txt)) {  
      us <- all_countries[all_countries$country_name == "United States", ]  
      if (nrow(us) > 0) {  
        return(list(country_id = us$country_id[1], country_name = "United States"))  
      }  
    }  
  }  
  
  # =====  
  # EXACT MATCH (original)  
  # =====  
  
  for (i in seq_len(nrow(all_countries))) {  
    cname <- tolower(all_countries$country_name[i])  
    pattern <- paste0("\\b", cname, "\\b")  
    if (grepl(pattern, txt)) {  
      return(list(  
        country_id = all_countries$country_id[i],  
        country_name = all_countries$country_name[i]  
      ))  
    }  
  }  
  
  # =====  
  # FUZZY MATCH (minimum 4 chars, no short words)  
  # =====  
  
  words <- unlist(strsplit(txt, "\\s+"))  
  best_dist <- Inf  
  best_match <- NULL  
  
  for (word in words) {  
    if (nchar(word) < 4) next  
    if (word %in% us_states) next
```


Prompt to AI models

The prompt that I gave to the AI models (Copilot, Gemini, Deepseek, Chat GPT)

I am building a shiny app for the information management course assignment. I need you help me to design this application,

I created my database in pgadmin with these tables:

dim_countries: for saving the country name, region and country id

dim_events: saving event_id, event_name, event_date, location, country_id

dim_date: date_id, month, day, year

dim_participants: person_id, first_name, last_name, country_id, organization

dim_topic: topic_id, topic_name, policy_area

fact_questions: question_id, person_id, question_text, created_at, topic_id, event_id, answer_text, date_id, country_id, status

dim_answers: answer_id, question_id, responder, answer_text, answered_at

embeddings: question_id, embedding

question_clusters: cluster_id, main_question, answer_text, created at

there are 3 pages in this application

Submit question page:

User enters questions, organization and name

AI model detects topic, country (if there is any country in the question) and typo and show the correct form of the word when I bring the curser on the wrong word to check the spelling. And give me suggestions for the possible models and approaches.

If detection fails do not show warnings or errors, put Unknown and let the question be submitted.

I need a person who is the admin of event, type the name of the even, date, and location and it shows on the top of the question section for the participants what the name of the event is.

Speaker page

In this page the speaker will see the question, the participant's name, the organization

Multiple selection is required if they want to select several questions and approve them to show in the display page

Approved selected questions go to the queue for display page

Live Display page

I need the speaker one question at a time then speak and the speech to text, type everything they say and when hit the next question, the answer submit in the database and show that this is the answer for that question.

Next question loads the next question in the queue

Do not show all questions when we come to display page

Action:

Give me clean, correct code

Fix issues

Explain where should I put each piece of codes

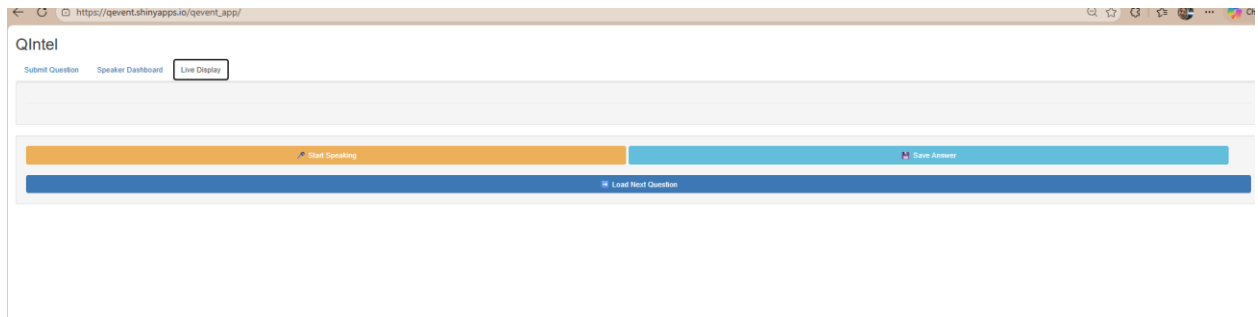
Pictures of the application:

The screenshot shows the QIntel application interface. At the top, there's a navigation bar with "Submit Question", "Speaker Dashboard", and "Live Display" links. Below this, the "Event Setup (Admin)" section is visible. It contains fields for "Event Name" (White House Press Briefing - May 2026), "Event Date" (2026-05-04), and "Location (optional)" (Washington, DC). A "Set / Update Active Event" button is at the bottom of this section. Below the event setup, a message states: "No active event set. Admin: please define the event above before participants submit questions." The "Your Question" section has a text input field with an example: "Example: What is the problem of jobs in Oklahoma?". Below this, the "Your Information" section includes fields for "Organization / Outlet (Required)" (Reuters, AP, CNN) and "Your Name (Optional)" (John Smith). A "Submit Question" button is at the bottom of the form.

Speaker Dashboard

The screenshot shows the QIntel application interface from the Speaker Dashboard perspective. The navigation bar at the top has "Submit Question", "Speaker Dashboard" (highlighted), and "Live Display" links. On the left, there are two buttons: "Refresh" and "Approve & Push to Live". The main area is titled "Pending Questions" and shows a "Show 10 entries" dropdown. A search bar is present. Below the search bar, there's a table with columns: ID, Question, Organization, Participant, and Time. The table currently displays "No data available in table". At the bottom, it says "Showing 0 to 0 of 0 entries" and has "Previous" and "Next" navigation links.

Live display



Appendix

These are my codes in app.r

Libraries:

```
library(shiny)
```

```
library(DBI)
```

```
library(RSQLite)
```

```
library(DT)
```

```
library(hunspell)
```

```
library(httr2)
```

```
library(jsonlite)
```

```
library(maps)
```

Server code:

```

# =====
# SERVER
# =====

server <- function(input, output, session) {

  state <- reactiveValues(
    active_event_id = NULL,
    active_event_name = NULL,
    active_event_date = NULL,
    cur_q = "",
    cur_org = "",
    cur_ans = "",
    current_question_id = NULL,
    refresh = 0,
    current_topic = "",
    lda_model_data = NULL
  )

  # Train topic model on existing questions when app starts
  observe({
    state$lda_model_data <- train_topic_model(con)
  })

  # -----
  # EVENT SETUP (ADMIN)
  # -----
  observeEvent(input$set_event, {
    req(input$event_name)

    ev_name <- trimws(input$event_name)
    ev_date <- as.Date(
      if (is.null(input$event_date)) Sys.Date() else input$event_date
    )
    ev_loc <- trimws(input$event_location)

    d_day <- as.integer(format(ev_date, "%d"))
    d_month <- as.integer(format(ev_date, "%m"))
    d_year <- as.integer(format(ev_date, "%Y"))

    existing_date <- dbGetQuery(
      con,
      "SELECT date_id FROM dim_date WHERE day = ? AND month = ? AND year = ?",
      params = list(d_day, d_month, d_year)
    )

    if (nrow(existing_date) == 0) {
      dbExecute(
        con,

```

```

837 - # -----
838 - # COUNTRY & TOPIC DETECTION (REAL-TIME)
839 - # -----
840 - observe({
841 -   req(input$question)
842 -   qtxt <- input$question
843 -   c_info <- detect_country_fuzzy(qtxt, con)
844 - })
845 -
846 - # -----
847 - # SUBMIT QUESTION
848 - # -----
849 - observeEvent(input$submit_btn, {
850 -   if (trimws(input$organization) == "") {
851 -     showNotification("organization is required.", type = "error")
852 -     return()
853 -   }
854 -
855 -   if (trimws(input$question) == "") {
856 -     showNotification("question cannot be empty.", type = "error")
857 -     return()
858 -   }
859 -
860 -   req(input$question, input$organization)
861 -
862 -   if (is.null(state$active_event_id)) {
863 -     showNotification(
864 -       "No active event set. Admin must define the event first.",
865 -       type = "error"
866 -     )
867 -     return()
868 -   }
869 -
870 -   tryCatch({
871 -     question_text <- trimws(input$question)
872 -     organization <- trimws(input$organization)
873 -     reporter_name <- trimws(input$reporter_name)
874 -     if (reporter_name == "") reporter_name <- "Anonymous"
875 -
876 -     c_info <- detect_country_fuzzy(question_text, con)
877 -     topic_label <- detect_topic_keywords(
878 -       question_text,
879 -       state$lda_model_data
880 -     )
881 -
882 -     if (is.na(topic_label) || topic_label == "" || topic_label == "General") {
883 -       topic_label <- "General"

```

```

1001 # -----
1002 # SPEAKER DASHBOARD
1003 # -----
1004 output$pending_table <- renderDT({
1005   state$refresh
1006
1007   data <- dbGetQuery(
1008     con,
1009     "SELECT
1010       fq.question_id,
1011       fq.question_text,
1012       dp.organization AS organization,
1013       (dp.first_name || ' ' || dp.last_name) AS participant_name,
1014       fq.created_at
1015     FROM fact_questions fq
1016     LEFT JOIN dim_participants dp
1017       ON fq.person_id = dp.person_id
1018     WHERE fq.status = 'pending'
1019     ORDER BY fq.created_at DESC"
1020   )
1021
1022   datatable(
1023     data,
1024     selection = "multiple",
1025     rownames = FALSE,
1026     colnames = c("ID", "Question", "Organization", "Participant", "Time"),
1027     options = list(pageLength = 10)
1028   )
1029 })
1030
1031 observeEvent(input$refresh, {
1032   state$refresh <- state$refresh + 1
1033   showNotification("Refreshed!", type = "message")
1034 })
1035
1036 observeEvent(input$approve, {
1037   req(input$pending_table_rows_selected)
1038
1039   df <- dbGetQuery(
1040     con,
1041     "SELECT
1042       fq.question_id,
1043       fq.question_text,
1044       dp.organization,
1045       TRIM(dp.first_name || ' ' || dp.last_name) AS participant_name
1046     FROM fact_questions fq
1047     JOIN dim_participants dp
1048       ON fq.person_id = dp.person_id
1049     WHERE fq.status = 'pending'"

```

462/3  (Untitled0) 

```

1073 # -----
1074 # LIVE DISPLAY
1075 # -----
1076
1077 output$live_q <- renderText(state$cur_q)
1078 output$live_org <- renderText(state$cur_org)
1079 output$live_answer <- renderText(state$cur_ans)
1080
1081 observeEvent(input$speech_text, {
1082   req(input$speech_text)
1083   state$cur_ans <- input$speech_text
1084 })
1085
1086 observeEvent(input$save_answer, {
1087   req(state$current_question_id)
1088   req(state$cur_ans)
1089
1090   if (trimws(state$cur_ans) == "") {
1091     showNotification("No answer detected.", type = "warning")
1092     return()
1093   }
1094
1095   dbExecute(
1096     con,
1097     "INSERT INTO dim_answers
1098       (question_id, responder, answer_text, answered_at)
1099     VALUES (?, ?, ?, CURRENT_TIMESTAMP)",
1100     params = list(state$current_question_id, "Speaker", state$cur_ans)
1101   )
1102
1103   dbExecute(
1104     con,
1105     "UPDATE fact_questions
1106     SET status = 'answered', answer_text = ?
1107     WHERE question_id = ?",
1108     params = list(state$cur_ans, state$current_question_id)
1109   )
1110
1111   showNotification("Answer saved!", type = "message")
1112 })
1113
1114 observeEvent(input$next_question, {
1115
1116   next_q <- dbGetQuery(
1117     con,
1118     "SELECT
1119       fq.question_id,
1120       fq.question_text,

```

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database											
Database Structure Browse Data Edit Pragmas Execute SQL											
Table: fact_questions Filter In any column											
	question_id	person_id	cluster_id	question_text	created_at	topic_id	event_id	answer_text	date_id	country_id	status
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	NULL	test question	2026-05-06 19:01:30	1	1	NULL	1	NULL	answering
2	2	2	NULL	What are the UTD plans for Graduate ...	2026-05-06 20:54:24	1	2	NULL	1	NULL	answering
3	3	3	NULL	What is your prediction for housing ...	2026-05-07 00:16:01	1	3	NULL	1	222	answering
4	4	4	NULL	What is the price of houses' ...	2026-05-07 00:17:02	1	3	NULL	1	222	queued
5	5	5	NULL	what is the next prediction for ...	2026-05-07 00:38:26	1	4	NULL	1	253	queued
6	6	6	NULL	what is the prediction for energy ...	2026-05-07 01:27:47	1	5	NULL	1	107	queued
7	7	7	NULL	what is the effect of Databases in ...	2026-05-07 02:09:53	1	6	NULL	1	222	queued
8	8	8	NULL	What is the impact of Covid 19 ...	2026-05-07 02:31:35	1	7	NULL	1	222	queued
9	9	9	NULL	what is the budget of schools in New...	2026-05-07 02:44:52	1	9	NULL	1	222	queued
10	10	10	NULL	What is the job problem in Ohio?	2026-05-07 02:45:45	1	9	NULL	1	222	queued
11	11	11	NULL	Germany problem	2026-05-07 11:58:45	2	12	NULL	2	57	queued
12	12	12	NULL	Russia Problem	2026-05-07 11:59:13	2	12	NULL	2	194	queued
13	13	13	NULL	Is there a problem with Ukraine too?	2026-05-07 11:59:55	2	12	NULL	2	236	queued
14	14	14	NULL	What is the plan for student ...	2026-05-07 12:03:59	3	13	NULL	2	222	queued
15	15	15	NULL	what is the future of Dallas Economy...	2026-05-07 12:30:39	1	15	NULL	2	222	queued
16	16	16	NULL	Dubai future what would be?	2026-05-07 14:05:38	1	21	NULL	2	52	queued
17	17	17	NULL	Do you expect more war in the United...	2026-05-07 14:06:32	1	21	NULL	2	1	queued
18	18	18	NULL	what is the problem in the Dubai, ...	2026-05-07 14:38:16	1	22	NULL	2	8	queued